

Forecast-Driven Energy-Aware Scheduling for Smart Manufacturing: A Predict-then-Optimize Framework with Quantile Forecasting and Constraint-Programming Refinement

Dr. S. Suresh Kumar^{*1}, Savithri R^{†1}, Hemanth Kumar D¹, Balaji R¹, and Bijulesh¹

¹Department of Artificial Intelligence and Data Science, Rajalakshmi Engineering College, Chennai, Tamil Nadu, India

Abstract

Rising energy costs and carbon-reporting obligations are pushing discrete manufacturing toward schedules that are aware not only of due dates and machine capacity but also of time-varying grid tariffs and emissions. This paper presents a complete predict-then-optimize software framework for smart-manufacturing scheduling: a data preprocessing pipeline, a multi-quantile XGBoost energy/carbon/power-factor forecaster, six scheduling algorithms culminating in a proposed tariff-aware robust dispatching heuristic (FD-PDTS) and a warm-started constraint-programming (CP-SAT) refinement stage, a twelve-metric KPI evaluation engine, and an interactive dashboard. We describe the system architecture, dataset schemas, feature engineering, the tariff-weighted quantile loss function, the scheduling formulations, and the engineering challenges encountered during development, in sufficient detail for independent reproduction. On a shared synthetic benchmark instance, the CP-SAT-refined schedule reduces energy cost by 32.4% and peak grid load by 42.4% relative to a first-come-first-served baseline, at the cost of a 73.1% increase in average waiting time. We report this as a preliminary, prototype-stage result: an independent audit of the pipeline identified an invalid recursive multi-step forecast, quantile crossing in 13.14% of test predictions, an unbounded power-factor output, and a unit inconsistency between kWh and kW quantities in the peak-load KPI. We document these issues explicitly, discuss the engineering challenges and solutions encountered while building the system, and outline the corrections and multi-instance validation required before the framework can be presented as validated industrial evidence.

Keywords: smart manufacturing; energy-aware scheduling; quantile forecasting; predict-then-optimize; constraint programming; XGBoost; demand response

1 Introduction

Manufacturing accounts for a substantial share of industrial electricity demand, and facilities that operate under time-of-use tariffs or grid demand charges can materially reduce operating cost and peak-load exposure by shifting flexible jobs toward lower-tariff, lower-emissions periods. This challenge sits within the broader transition toward Industry 4.0 and smart manufacturing, in which cyber-physical systems and data-driven modeling are reshaping production planning and control

^{*}Professor; corresponding author.

[†]Assistant Professor.

[20, 21]. Traditional manufacturing scheduling systems often rely on simple heuristics such as first-come-first-served (FCFS) and earliest-deadline-first (EDF) without considering dynamic energy tariff structures and real-time energy availability predictions. This leads to several well-documented inefficiencies: suboptimal energy costs from scheduling jobs during peak-tariff periods, missed opportunities to leverage off-peak energy rates, an inability to adjust schedules based on predicted energy availability, and unnecessary environmental impact through excessive energy consumption during carbon-intensive grid periods.

Energy-efficient and energy-aware scheduling has therefore become an active research area within production and operations management, spanning mixed-integer formulations, heuristic dispatching rules, and multi-agent architectures that couple production schedules to on-site or grid energy signals. Much of this literature assumes that near-future energy prices, machine power draw, or emissions factors are known with reasonable certainty; in practice, these quantities must themselves be forecast, and the quality of the forecast directly bounds the quality of any schedule built on top of it.

The *predict-then-optimize* paradigm formalizes this coupling: a predictive model produces estimates of otherwise-unknown parameters, and a downstream optimizer consumes those estimates as if they were known coefficients of the scheduling problem. This framing is attractive for smart manufacturing because it allows a forecasting model, such as a gradient-boosted tree ensemble trained on historical energy meter data, to be combined with a combinatorial scheduler, such as constraint programming, without requiring an end-to-end differentiable model of the entire pipeline.

This paper documents a complete implementation of such a framework, applied to a synthetic but structurally realistic smart-manufacturing scenario built around steel-industry energy consumption data. Unlike a purely algorithmic treatment, we describe the system as an engineered artifact: its module organization, data schemas, configuration management, machine-availability modeling, and the concrete engineering challenges that arose during development and how they were resolved. We then report both the scheduling-side results and an independent audit of the forecasting stage, and we are explicit about where the current implementation falls short of a validated industrial system.

1.1 Problem Statement

Traditional manufacturing scheduling systems often rely on simple heuristics (FCFS, EDF) without considering dynamic energy tariff structures and real-time energy availability predictions. This leads to suboptimal energy costs due to scheduling jobs during peak-tariff periods, missed opportunities to leverage off-peak energy rates, an inability to adjust schedules based on predicted energy availability, and avoidable environmental impact through excessive energy consumption during high-carbon-intensity periods.

1.2 Contributions

Our contributions are:

1. A forecast-driven scheduling architecture coupling tariff-weighted quantile XGBoost forecasting of energy, carbon, and power factor with six scheduling strategies, including a proposed robust dispatching heuristic (Forecast-Driven Priority Dispatching with Tariff Shifting, FD-PDTS) and a CP-SAT refinement stage.
2. A complete, reproducible description of the data schemas, feature engineering, configuration management, and machine-availability modeling used throughout the pipeline.
3. A twelve-metric KPI evaluation framework spanning economic, operational, compliance, environmental, and quality dimensions, together with explicit fair-accounting rules for energy

charged beyond the forecast horizon.

4. A candid account of the engineering challenges encountered during development – machine availability modeling, tariff structure integration, forecast horizon boundaries, objective-function serialization, data quality, feature leakage, and hyperparameter tuning – and the solutions adopted for each.
5. A transparent, independently audited account of the pipeline’s current validity limitations, together with a concrete improvement plan, so that the reported cost and peak-load reductions are correctly understood as preliminary, prototype-stage findings rather than validated industrial evidence.

2 Related Work

Elmachtoub and Grigas [1] formalized the *smart predict-then-optimize* framework, in which a predictive model is trained with a loss function that directly accounts for the downstream optimization task rather than pure prediction accuracy. Their work established the theoretical basis for coupling forecasting and optimization stages, which motivates the architecture proposed in this paper, where a quantile forecasting model feeds directly into a scheduling optimizer rather than being trained and evaluated in isolation.

Gahm et al. [3] conducted a comprehensive review of energy-efficient scheduling in manufacturing companies, categorizing prior contributions by decision level and energy objective and proposing a research framework for integrating energy awareness into classical scheduling formulations. Their review highlighted that most existing energy-aware scheduling studies assume energy prices and consumption profiles are known in advance, a gap that the predict-then-optimize approach in this paper is designed to address directly by making the forecast an explicit, auditable stage of the pipeline rather than a background assumption.

Fang et al. [4] introduced a scheduling approach explicitly targeting power consumption and carbon-footprint reduction in manufacturing, extending classical scheduling objectives such as makespan to include environmental criteria. Their formulation demonstrated that carbon- and energy-aware objectives can be incorporated into scheduling models without abandoning conventional operational constraints, an approach reflected in the multi-objective, twelve-metric KPI evaluation used in this work.

Chen and Guestrin [2] developed XGBoost, a scalable gradient-boosted tree-boosting system, which underlies the quantile forecasting models used in this paper. Its regularized objective, efficient handling of sparse tabular data, and support for custom, user-supplied gradient and Hessian functions make it well suited to short-horizon energy-load forecasting from meter data and to the tariff-weighted quantile objective described in Section 6.

Koenker and Bassett [5] established the statistical foundation of quantile regression, which this paper adapts, via a custom pinball-loss objective, to produce calibrated p10/p50/p90 forecasts of energy usage, carbon emissions, and power factor rather than single-point estimates. Their asymmetric loss formulation is the mathematical basis for the tariff-weighted objective implemented as a custom XGBoost objective in this work.

Bertsimas and Sim [6] proposed a tractable robust-optimization framework, *the price of robustness*, that controls the conservatism of a solution against parameter uncertainty via an explicit uncertainty budget. Their formalism is used in this paper as the benchmark against which the current heuristic robustness bias in the proposed FD-PDTS scheduler is compared, and as the target for a future, mathematically explicit robust reformulation, since the present robust variant uses a fixed empirical tariff-percentile bias rather than a formally derived uncertainty set.

The constraint-programming refinement stage in this work is implemented using Google’s OR-Tools CP-SAT solver [7], a widely used open-source constraint-programming and combinatorial-

optimization toolkit that supports warm-starting from an externally supplied feasible solution, which this paper exploits by warm-starting CP-SAT from the FD-PDTS heuristic’s schedule.

Ham et al. [8] proposed both mixed-integer and constraint-programming formulations for flexible job shop scheduling with a machine shutdown (on/off) strategy aimed at minimizing makespan and total energy consumption jointly, and showed through practical operational scenarios that energy savings need not come at the expense of productivity. This supports the design choice, in this paper, of framing energy cost, waiting time, and tardiness as jointly optimized rather than strictly competing objectives in the CP-SAT stage.

Park and Ham [9] studied energy-aware flexible job shop scheduling explicitly under time-of-use electricity pricing, developing exact and heuristic solution methods that shift flexible operations toward lower-price intervals subject to machine and precedence constraints. Their time-of-use framing closely parallels the tariff-weighted forecasting and dispatching bias used in the FD-PDTS heuristic proposed in this paper, and their exact-versus-heuristic comparison motivates the inclusion of a warm-started constraint-programming refinement stage rather than relying on the heuristic alone.

Terbrack and Claus [10] presented a generalized energy-aware flexible job shop scheduling model solved via constraint programming that simultaneously targets real-time-pricing energy cost, peak demand, and emissions, while incorporating sequence-dependent setup times and due-date compliance; their computational study over twenty problem instances found average reductions of 5.3% in energy cost, 11.8% in peak demand, and 8.3% in emissions relative to traditional scheduling, and explicitly noted that these objectives can partly conflict. Their multi-instance evaluation methodology, and their finding that setup and due-date constraints materially limit achievable energy savings, directly motivate two of the corrections identified in Section 13 of this paper: benchmarking across many instances rather than one, and modeling setup/changeover constraints consistently across every scheduling strategy compared.

2.1 Flexible Job Shop Scheduling: Reviews and Formulations

The general flexible job shop scheduling problem underlying this work has been extensively reviewed. Xie et al. [11] surveyed solution approaches for flexible job shop scheduling, classifying methods into exact algorithms, heuristics, and metaheuristics and highlighting the growing role of hybrid approaches. More recently, Dauzère-Pérès et al. [12] provided an extensive review covering makespan and alternative/multi-criteria objectives, a wide range of problem-extending constraints, and a three-field notation for classifying the literature; both reviews confirm that energy- and tariff-aware objectives, of the kind this paper addresses, remain a comparatively less mature branch of the broader flexible job shop literature relative to makespan-centric formulations. On the exact-formulation side, Meng et al. [13] developed both mixed-integer linear programming and constraint-programming formulations for the distributed flexible job shop scheduling problem, providing a direct methodological precedent for the CP-SAT formulation used in the hybrid refinement stage of this paper (Section 7.8).

2.2 Reinforcement Learning for Energy-Aware Scheduling

A rapidly growing, complementary line of work applies deep reinforcement learning (DRL) to energy-aware flexible job shop scheduling, generally learning a dispatching policy end to end rather than combining a forecaster with a heuristic or solver. Rui et al. [14] proposed a graph reinforcement learning method for flexible job shop scheduling under industrial demand response, jointly optimizing load-side and grid-side energy metrics via heterogeneous graph neural network schedulers. Song et al. [15] combined graph neural networks with deep reinforcement learning

for general flexible job shop scheduling, a representation later adapted by several energy-aware variants. Tang and Dong [16], Zhao and Deng [17], Chang et al. [18], and Huang et al. [19] each proposed graph- or attention-based deep reinforcement learning schedulers for variants of the flexible job shop problem, collectively illustrating a shift toward learned, adaptive dispatching policies as an alternative to the heuristic-plus-solver architecture used in this paper. Zhang et al. [22] and Modrák et al. [23] each provide systematic reviews of reinforcement-learning-based scheduling for sustainable and resilient manufacturing, both noting that DRL schedulers can adapt to evolving conditions without hand-tuned heuristics but typically require substantially more training data and offer less interpretability than the explicit tariff-weighted forecasting and warm-started constraint-programming approach used in this paper. We identify a learned dispatching policy of this kind as a candidate long-term extension of the present architecture in Section 19.

2.3 Energy Forecasting

Hong et al. [24] reviewed the state of energy forecasting broadly, covering point, probabilistic, and hierarchical forecasting methods and identifying probabilistic forecasting – of the kind produced by the quantile XGBoost models in this paper – as an increasingly central requirement for forecast-informed decision-making under uncertainty, directly motivating the multi-quantile architecture described in Section 6.

2.4 Constraint Programming and Optimization

3 System Architecture

3.1 High-Level Design

The proposed system is organized as a four-phase orchestration pipeline. Phase 1 performs data ingestion, cleaning, and feature engineering. Phase 2 trains and applies multi-quantile XGBoost models to forecast energy usage, carbon emissions, and power factor. Phase 3 generates six candidate schedules using the algorithms described in Section 7. Phase 4 computes the twelve-metric KPI set described in Section 8 and produces the comparison reports consumed by the dashboard described in Section 9.

3.2 Technology Stack

Table 1 summarizes the technologies used in each layer of the system.

Table 1: Technology stack by system component.

Component	Technology
Data processing	Pandas, NumPy
ML / forecasting	XGBoost, scikit-learn
Optimization	Google OR-Tools (CP-SAT)
Frontend	Streamlit, Plotly
Serialization	Joblib
Configuration	Centralized Python configuration class
Logging	Python standard logging module

3.3 Module Organization

The codebase is organized into a backend package (data loading, cleaning, feature engineering, forecasting, scheduling, KPI computation, and synthetic data generation), a frontend package (the Streamlit dashboard and its multi-page structure), a data directory (raw and processed inputs), an output directory (generated schedules and reports), and a top-level orchestration script that sequences the four pipeline phases:

```
project_root/
|-- backend/
| |-- config.py # Global configuration parameters
| |-- preprocessing.py # Loading, cleaning, feature engineering
| |-- forecasting.py # Multi-quantile XGBoost models
| |-- scheduler.py # Six scheduling algorithms
| |-- kpi_calculator.py # KPI computation engine
| |-- utils.py # Logging, helper functions
| |-- generate_dataset.py # Synthetic data generation
| '-- models/ # Trained model artifacts
|-- frontend/
| |-- app.py # Main Streamlit dashboard
| |-- pages/ # Multi-page dashboard structure
| '-- utils/loader.py # Data loading utilities for frontend
|-- data/
| |-- raw/ # Raw CSV input files
| '-- processed/ # Cleaned and engineered datasets
|-- output/ # Generated schedules and reports
|-- run_pipeline.py # Main orchestration script
'-- requirements.txt # Python dependencies
```

3.4 Architecture Diagram

Figure 1 illustrates the end-to-end architecture. The two data sources – an industrial energy dataset and a synthetic job/machine dataset – feed preprocessing and feature engineering. The engineered features drive a quantile XGBoost forecasting module, whose outputs, together with the job and machine dataset, are consumed by the scheduling engine. Each generated schedule is scored by the KPI calculator and surfaced through the dashboard.

4 Data Sources and Schema

Three primary data sources feed the pipeline: an industrial energy dataset, a job table, and a machine table.

4.1 Energy Dataset Schema

The energy dataset is drawn from steel-industry energy consumption records. Table 2 lists its key columns.

4.2 Job Dataset Schema

Table 3 lists the job-table schema used by the scheduling module.

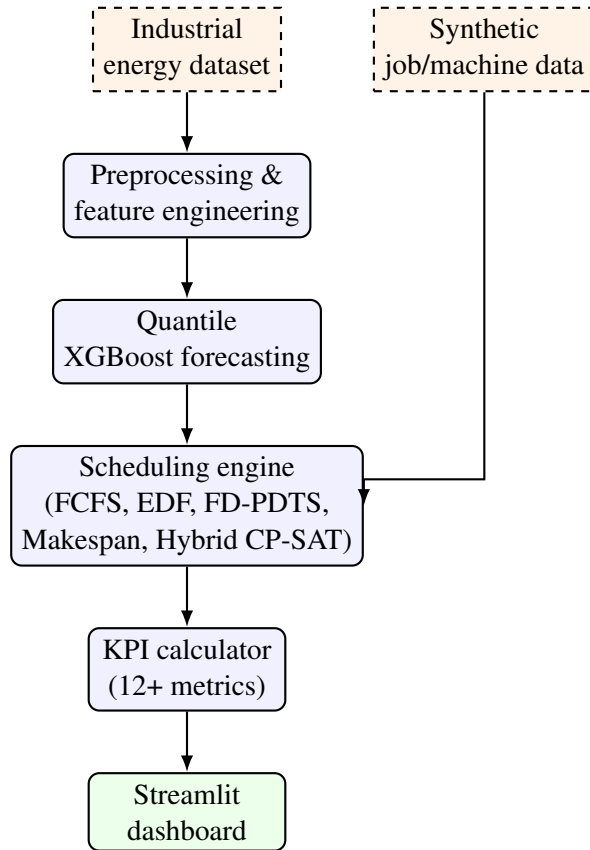


Figure 1: End-to-end pipeline architecture.

Table 2: Energy dataset schema.

Column	Description
date	Timestamp of the energy reading
Usage_kWh	Active energy consumption
Lagging_Current_Reactive_Power_kVarh	Reactive power component
Leading_Current_Reactive_Power_kVarh	Leading power component
CO2 (tCO2)	Carbon emissions coefficient
Lagging_Current_Power_Factor	Power factor metric
Leading_Current_Power_Factor	Leading power factor
WeekStatus	Weekday vs. weekend classification
Day_of_week	Numerical day indicator
Load_Type	Categorical: Maximum, Medium, or Light Load

Table 3: Job dataset schema.

Column	Description
Job_ID	Unique job identifier
Duration_min	Required processing time, in minutes
Deadline	Hard deadline for job completion
Priority	Job priority level for scheduling
Setup_Type	Job type used for changeover tracking

4.3 Machine Dataset Schema

Table 4 lists the machine-table schema.

Table 4: Machine dataset schema.

Column	Description
Machine_ID	Unique machine identifier
Idle_Power_kW	Standby power consumption
Active_Power_kW	Power consumption during job execution
Setup_Energy_kW	Energy consumed during setup / changeover
Maintenance_Window	Available operating hours

4.4 Dataset Statistics

Table 5 summarizes the scale of the energy and job/machine datasets used for the experiments reported in Section 12. The energy dataset contains 35,040 fifteen-minute records spanning calendar year 2018, with no missing values and no duplicate rows across 11 columns. The job/machine dataset comprises 180 jobs distributed across 8 machines with an average processing time of approximately 62.75 minutes; 75% of job arrivals fall within peak tariff hours, and each job is compatible with two to four of the eight machines.

Table 5: Summary statistics for the energy and job/machine datasets.

Energy dataset	Value
Records	35,040
Sampling interval	15 minutes
Coverage	1 year (2018)
Columns	11
Missing / duplicate rows	0 / 0
Light load mean usage	8.63 kWh
Medium load mean usage	38.45 kWh
Maximum load mean usage	59.27 kWh
Job/machine dataset	Value
Jobs	180
Machines	8
Total processing slots	753
Avg. processing time	62.75 min
Peak-hour job arrivals	75%
Priority classes (H/M/L)	34 / 101 / 45
Setup types	3
Compatible machines per job	2–4

5 Data Processing Pipeline

5.1 Data Cleaning

A dedicated data-cleaning stage validates and normalizes each of the three data sources. For the energy data, cleaning removes duplicate records, converts timestamps to a consistent datetime format (with an explicit day-first convention to resolve date-format ambiguity), coerces numeric columns to floating-point type, converts categorical columns to a categorical type, applies forward- and backward-fill imputation for missing values, and sorts records by timestamp to guarantee temporal consistency. For the job data, cleaning removes duplicate job identifiers (keeping the first occurrence) and converts duration, deadline, and priority fields to numeric types, imputing missing values where necessary. For the machine data, cleaning removes duplicate machine identifiers and coerces power-related fields to numeric values.

5.2 Feature Engineering

Energy consumption exhibits strong cyclical patterns at multiple time scales. Calendar features (hour of day, day of month, month, day of week) are extracted directly, and cyclical features are additionally encoded using sine/cosine transformations to preserve continuity across period boundaries (e.g., so that hour 23 and hour 0 are recognized as adjacent):

$$\text{hour_sin} = \sin\left(\frac{2\pi \cdot \text{hour}}{24}\right), \quad \text{hour_cos} = \cos\left(\frac{2\pi \cdot \text{hour}}{24}\right) \quad (1)$$

$$\text{month_sin} = \sin\left(\frac{2\pi \cdot (\text{month} - 1)}{12}\right), \quad \text{month_cos} = \cos\left(\frac{2\pi \cdot (\text{month} - 1)}{12}\right) \quad (2)$$

Lagged features provide historical context: a 1-step lag captures the immediately preceding 15-minute interval, a 2-step lag captures two intervals prior, a 4-step lag captures a one-hour-prior pattern, and a 96-step lag captures the corresponding interval 24 hours prior, exposing day-to-day consistency. Rolling statistics are computed over both a short window (4 intervals, i.e., one hour) and a long window (96 intervals, i.e., 24 hours), including rolling mean and rolling standard deviation, to characterize short-term trend/volatility and long-term trend/volatility respectively.

5.3 Train/Validation/Test Split Strategy

Data is split chronologically – not randomly – into training (the first 60% of historical data), validation (the next 20%, used for hyperparameter tuning), and test (the final 20%, used for performance evaluation) partitions. This chronological split respects the temporal nature of forecasting and avoids the lookahead bias that a random split would introduce, since a random split would allow the model to be validated on intervals that occur chronologically before intervals used in training.

6 Energy Forecasting Component

6.1 Multi-Quantile Forecasting Architecture

The forecasting system targets three physical quantities – active energy usage (`Usage_kWh`), carbon emissions (`CO2 (tCO2)`), and power factor (`Lagging_Current_Power_Factor`) – each predicted at one or more quantile levels. Energy usage and carbon emissions are each predicted at the 10th, 50th, and 90th percentiles (p10/p50/p90), spanning a conservative, median, and optimistic

estimate respectively, while power factor is predicted only at the median (p50), since it exhibits comparatively less variability. This yields seven independently trained XGBoost regressors in total.

6.2 Tariff-Weighted Sample Weighting

Energy costs vary significantly by load type, and forecast accuracy during high-tariff periods matters more for downstream scheduling decisions than accuracy during low-tariff periods. The system therefore implements tariff-weighted learning, in which training samples are weighted according to their grid demand category, as summarized in Table 6.

Table 6: Tariff-based sample weighting used during forecast training.

Load type	Weight	Rationale
Maximum Load	3.0	Peak periods: highest cost, most important to forecast accurately
Medium Load	1.5	Mid-range periods: moderate cost impact
Light Load	1.0	Off-peak periods: baseline importance

6.3 Custom XGBoost Objective

Standard regression treats all prediction errors symmetrically. The system instead implements a custom `TariffWeightedQuantileObjective` that computes an asymmetric, tariff-weighted gradient. For a target quantile $\tau \in (0, 1)$, true value y_i , and prediction $\hat{y}_i$, the tariff-weighted pinball loss minimized during training is

$$\mathcal{L}_\tau = \sum_{i=1}^n w_i \cdot \begin{cases} \tau (y_i - \hat{y}_i), & y_i \geq \hat{y}_i \\ (1 - \tau) (\hat{y}_i - y_i), & y_i < \hat{y}_i \end{cases} \quad (3)$$

where $w_i \propto c_i$ is a sample weight proportional to the electricity tariff c_i in effect at interval i (Table 6), so that errors made during high-tariff periods are penalized more heavily than errors made during low-tariff periods. Concretely, the objective supplies XGBoost with an asymmetric gradient

$$g_i = \begin{cases} -\tau \cdot w_i, & y_i \geq \hat{y}_i \\ (1 - \tau) \cdot w_i, & y_i < \hat{y}_i \end{cases}$$

and a strictly positive Hessian $h_i = w_i$, which is required for numerical stability of XGBoost’s second-order boosting updates. This asymmetry reflects an asymmetric real-world cost structure: under-predicting usage during a peak period represents a missed scheduling opportunity (costly), while over-predicting usage during an off-peak period simply results in conservative, still-safe scheduling.

6.4 Hyperparameters

Table 7 lists the XGBoost hyperparameters used for all seven forecasting models. A moderate tree depth (5) limits overfitting given the feature set size; a low learning rate (0.05) favors careful, incremental learning over many boosting rounds; row and column subsampling (0.8 each) provide additional regularization; and a fixed random seed (42) ensures reproducibility.

Table 7: XGBoost hyperparameters used for forecasting.

Hyperparameter	Value
n_estimators	300
max_depth	5
learning_rate	0.05
subsample	0.8
colsample_bytree	0.8
random_state	42
n_jobs	-1 (all available cores)

6.5 Serialization Considerations

Custom objective functions with internal state must be defined as top-level classes, not as classes nested inside a factory function, in order to be serializable by Joblib for later reuse. A nested class definition causes silent or cryptic pickling failures in production, since Python’s default pickling protocol cannot resolve a reference to a class defined inside another function’s local scope. The `TariffWeightedQuantileObjective` class is therefore defined at module level.

6.6 Prediction Output

For a new feature matrix, the trained forecaster returns predictions for `predicted_Usage_p10`, `predicted_Usage_p50`, `predicted_Usage_p90`, `predicted_CO2_p10`, `predicted_CO2_p50`, `predicted_CO2_p90`, and `predicted_PF_p50`, together with the associated `Load_Type` category used for tariff lookup by the scheduling module.

7 Scheduling Algorithm Framework

7.1 Problem Formulation

Given a job set $J = \{1, \dots, n\}$ with durations and deadlines, a machine set $M = \{1, \dots, m\}$, and a (forecasted) energy tariff $T(t)$ for each time slot t , the scheduling problem is to assign each job to a machine and start time so as to minimize a weighted combination of energy cost, makespan, and tardiness, subject to deadline constraints (or explicit tardiness accounting where deadlines cannot be met), machine-exclusivity constraints (each job on exactly one machine, no two jobs on the same machine overlapping in time), and setup/changeover constraints between incompatible job types on the same machine.

For the CP-SAT refinement stage specifically, with job j ’s start time s_j , completion time f_j , deadline d_j , and forecast-driven energy cost E_j , the minimized objective is

$$\min \alpha \sum_{j \in J} E_j + \beta \sum_{j \in J} (s_j - a_j) + \gamma \sum_{j \in J} \max(0, f_j - d_j) \quad (4)$$

subject to $f_j = s_j + p_j$ for processing time p_j , $s_j \geq a_j$ for arrival time a_j , machine assignment to a compatible machine $m \in M_j$, and pairwise non-overlap of any two jobs assigned to the same machine, where α , β , and γ are configured weights on cost, waiting time, and tardiness respectively. The current implementation of Eq. 4 omits direct terms for carbon emissions, peak load, and power-factor penalties even though configuration weights are defined for them; extending the objective to include these terms directly is identified as future work in Section 19.

7.2 Algorithm Overview

Table 8 summarizes the six implemented scheduling algorithms, their asymptotic complexity, and their key distinguishing feature.

Table 8: Overview of the six implemented scheduling algorithms.

Algorithm	Complexity	Key feature
FCFS	$O(n \log m)$	Simple, tariff-unaware queue
EDF	$O(n \log m)$	Deadline-priority baseline
FD-PDTS Deterministic	$O(n^2 m)$	Cost-aware, uses p50 forecast
FD-PDTS Robust	$O(n^2 m)$	Cost-aware, uses p10/p90 bands
Makespan-oriented	$O(n \log m)$	Minimizes job completion time
Hybrid (warm-start + CP-SAT)	$O(n^2 m + \text{solver})$	Warm-start plus constraint refinement

7.3 FCFS (First-Come-First-Served)

Jobs are sorted by arrival time. For each job in order, the machine offering the earliest available slot is selected and the job is scheduled immediately. FCFS is simple and predictable but performs no cost optimization, resulting in inefficient energy usage and, on tightly constrained instances, deadline misses.

7.4 EDF (Earliest-Deadline-First)

Jobs are sorted by deadline in ascending order. For each job in order, the algorithm searches for a machine on which the job can be completed before its deadline, scheduling it at the earliest available slot on that machine. EDF is a strong baseline for deadline compliance but ignores energy costs entirely, providing no tariff awareness.

7.5 FD-PDTS Deterministic

The core algorithmic contribution of this work is Forecast-Driven Priority Dispatching with Tariff Shifting (FD-PDTS). The deterministic variant proceeds as follows:

1. Compute a priority score for each job, combining deadline slack with a configured priority weight:

$$\text{Priority}_j = \frac{\text{Deadline}_j - \text{Duration}_j}{\max_k(\text{Deadline}_k)} + \text{PriorityWeight}_j. \quad (5)$$

2. Build a tariff array from the p50 (median) energy forecast over the full scheduling horizon.
3. Process jobs in descending priority order. For each job, evaluate every compatible machine, computing on each the earliest feasible slot (accounting for changeover time from the previous job on that machine) and the associated energy cost

$$\text{Cost}_{\text{job}} = \sum_{t=\text{start}}^{\text{start}+\text{duration}} \text{ActivePower} \times \text{Tariff}(t) + \text{SetupPower} \times \text{Tariff}(\text{start}). \quad (6)$$

4. Select the machine/slot combination with minimum energy cost and update that machine's busy-interval calendar accordingly.

This deterministic variant considers the dynamic tariff structure explicitly and schedules jobs during low-cost periods whenever feasible, in contrast to FCFS and EDF.

7.6 FD-PDTS Robust

The robust variant enhances FD-PDTS by using quantile forecast bands rather than a single point estimate. Three separate tariff arrays are built from the p10, p50, and p90 forecasts. For each candidate machine/slot placement, the algorithm computes a cost band: a minimum-cost scenario (p10 tariffs), an expected-cost scenario (p50 tariffs), and a maximum-cost, worst-case scenario (p90 tariffs). The decision rule selects the machine/slot minimizing the worst-case (p90) cost, providing a hedge against forecast uncertainty at the expense of some average-case cost efficiency relative to the deterministic variant.

7.7 Makespan-Oriented Baseline

This baseline sorts jobs by duration in descending order (a longest-processing-time, LPT-style heuristic) and, for each job, assigns it to the machine offering the earliest completion time, with no consideration of energy tariffs. Its purpose is to minimize operational time and enable faster throughput, serving as a tariff-unaware speed baseline against which the energy-aware methods can be compared.

7.8 Hybrid (Warm-Start + CP-SAT Refinement)

The hybrid method proceeds in two phases. In the warm-start phase, the FD-PDTS deterministic algorithm generates an initial, already energy-cost-conscious feasible solution. In the refinement phase, this solution is supplied as a warm start to a Google OR-Tools CP-SAT model with decision variables for each job's start time, machine assignment, and end time, subject to temporal consistency ($\text{end}_j = \text{start}_j + \text{duration}_j$), machine-capacity (no overlapping jobs on the same machine), deadline, and changeover-precedence constraints, minimizing a weighted combination of energy cost and schedule variance within a fixed 60-second solver time limit. This mixed-integer/constraint-programming pairing follows a methodological precedent established for the distributed flexible job shop scheduling problem by Meng et al. [13]. Because the solver begins from an already-good feasible schedule rather than searching from scratch, it can spend its limited time budget on refinement rather than initial feasibility discovery.

7.9 Machine Calendar: Interval-Based Availability Tracking

Rather than modeling each machine as having a single "free pointer" (as in a simple processor-queue model), the system tracks machine availability as an explicit list of busy intervals via a `MachineCalendar` class. Real manufacturing machines frequently exhibit multi-shift schedules with daytime gaps – for example, a machine may run a night shift, sit idle during the day due to other production-line commitments, and resume in the evening. An interval-based calendar correctly identifies and fills such gaps, whereas a single-pointer model would incorrectly treat the machine as unavailable until the end of its most recent job. The calendar exposes an `earliest_fit` operation that finds the earliest contiguous free gap of sufficient duration on or after a job's arrival time, accounting for changeover time implied by the setup type of the immediately preceding job.

7.10 Formal Pseudocode

Algorithm 1 formalizes the end-to-end scheduling pipeline, and Algorithm 2 details the per-job scoring rule used by the FD-PDTS heuristic (in both its deterministic and robust forms) when selecting a machine and timing offset for each job.

Algorithm 1 End-to-end forecast-driven scheduling pipeline

Require: Energy series E , job set J , machine set M

Ensure: Six candidate schedules and their KPI reports

```

1:  $E' \leftarrow \text{Clean}(E)$ ; parse timestamps (day-first)
2:  $F \leftarrow \text{EngineerFeatures}(E')$  ▷ lags, rolling stats, cyclic
3:  $(E'_{\text{train}}, E'_{\text{val}}, E'_{\text{test}}) \leftarrow \text{ChronologicalSplit}(E', 60/20/20)$ 
4: for each quantile target  $\tau \in \{p10, p50, p90\}$  and variable  $v \in \{\text{energy}, \text{CO}_2, \text{PF}\}$  do
5:   Train XGBoost model  $\hat{y}_{v,\tau}$  on  $E'_{\text{train}}$  using Eq. 3
6: end for
7:  $\hat{Y} \leftarrow \text{Forecast}(\hat{y}_{v,\tau}, \text{horizon} = 48\text{h})$ 
8: for each strategy  $s \in \{\text{FCFS}, \text{EDF}, \text{FD-PDTS Det.}, \text{FD-PDTS Robust}, \text{Makespan}, \text{Hybrid}\}$  do
9:    $\text{schedule}_s \leftarrow \text{Dispatch}(J, M, \hat{Y}, s)$ 
10:  if  $s = \text{Hybrid}$  then
11:     $\text{schedule}_s \leftarrow \text{WarmStartRefine}(\text{schedule}_{\text{FD-PDTS Det.}})$  via Eq. 4
12:  end if
13:   $\text{KPI}_s \leftarrow \text{EvaluateKPI}(\text{schedule}_s)$ 
14: end for
15: return  $\{\text{schedule}_s, \text{KPI}_s\}$  for all  $s$ 

```

8 Key Performance Indicator (KPI) System

The system computes twelve or more KPIs across five dimensions: economic, operational, compliance, environmental, and quality.

8.1 Economic KPIs

Total energy cost sums the energy charges for all scheduled jobs and idle periods:

$$\text{Cost} = \sum_j (\text{ActivePower}_j \times \text{Duration}_j + \text{SetupPower}_j) \times \sum_{t \in \text{job}} \text{Tariff}(t). \quad (7)$$

Peak hour load is the maximum simultaneous power draw across all machines, identifying grid-stress periods and potential demand charges.

8.2 Operational KPIs

Makespan is the elapsed time from the first job's start to the last job's completion, computed as $\max(\text{EndSlot}) - \min(\text{StartSlot})$. Machine utilization is the percentage of total available machine time spent on jobs:

$$\text{Utilization} = \frac{\sum_j \text{JobDuration}_j}{|M| \times \text{Makespan}} \times 100\%. \quad (8)$$

Algorithm 2 FD-PDTS job scoring (deterministic when $r = \text{false}$, robust when $r = \text{true}$)

Require: Job j with arrival a_j , deadline d_j ; compatible machines M_j ; forecast $\hat{Y}$; robustness flag r

Ensure: Selected machine m^* and start time s_j^*

```
1: best_score  $\leftarrow +\infty$ 
2: for each candidate machine  $m \in M_j$  do
3:   for each candidate start time  $t$  in feasible window do
4:      $c \leftarrow \hat{Y}_{p50}(t)$ 
5:     if  $r$  then ▷ robust variant only
6:        $c \leftarrow \hat{Y}_{p90}(t)$  ▷ worst-case tariff bias
7:     end if
8:     wait  $\leftarrow t - a_j$ ; tardy  $\leftarrow \max(0, t + p_j - d_j)$ 
9:     score  $\leftarrow c + \lambda_1 \cdot \text{wait} + \lambda_2 \cdot \text{tardy} + \lambda_3 \cdot \text{PeakGuard}(t)$ 
10:    if score < best_score then
11:      best_score  $\leftarrow$  score;  $(m^*, s_j^*) \leftarrow (m, t)$ 
12:    end if
13:  end for
14: end for
15: return  $(m^*, s_j^*)$ 
```

Average waiting time is the mean delay between a job's arrival and the start of its processing, and total idle time is the cumulative time machines sit idle across the schedule, which is associated with idle-power costs.

8.3 Compliance KPIs

On-time completion is the percentage of jobs meeting their deadlines:

$$\text{OTC} = \frac{|\{j : \text{EndTime}_j \leq \text{Deadline}_j\}|}{|J|} \times 100\%. \quad (9)$$

Total delay is the cumulative lateness across all jobs exceeding their deadlines, and the count of late jobs indicates how many jobs missed their deadlines outright.

8.4 Environmental KPIs

Total carbon emissions sum the CO₂ contribution of every scheduled slot:

$$\text{CO}_2 = \sum_{\text{slots}} \text{CO}_2\text{Rate}(t) \times \text{ActivePower}(t), \quad (10)$$

and carbon intensity per job (tCO₂/job) is the average emissions per job, enabling comparison across scheduling methods independent of total job count.

8.5 Quality KPIs

Total power-factor penalty captures the cost incurred from a non-unity power factor:

$$\text{PFPenalty} = \sum_t |\text{PF}(t) - 1.0| \times \text{PenaltyRate} \times \text{Power}(t). \quad (11)$$

A schedule robustness score additionally measures the buffer time between job completions and their deadlines, with a higher score indicating a more resilient schedule against small perturbations.

8.6 Fair Accounting Rules

Three accounting rules are enforced for scientific and economic fairness. First, jobs that extend beyond the 48-hour forecast window are charged using a cyclically repeated forecast pattern, rather than being assigned zero cost, which would otherwise create a perverse incentive to defer work indefinitely far into the future. Second, idle power is charged for the entire makespan duration on every machine, not truncated at the forecast-window boundary, again to avoid an incentive to schedule work arbitrarily far away. Third, all time periods, including night shifts, receive consistent tariff treatment, with no artificial zero-cost periods.

9 Dashboard and Visualization

9.1 Design Philosophy

The dashboard is designed around the principle of "executive intelligence at a glance": prominent top-level KPI cards immediately surface scheduler performance differences, drill-down views allow detailed per-schedule analysis, comparative visuals place algorithms side by side, and a dark, professional theme is used throughout.

9.2 Dashboard Sections

The home page presents an executive summary with key metrics, a schedule-comparison overview, and interactive filters for schedule selection. The KPI page presents a detailed KPI comparison across all six scheduling algorithms, with bar charts, gauge charts, and trend lines, and an exportable KPI summary table. The scheduling page presents a per-machine Gantt chart showing job placement over time, a machine timeline view, and a job-details table with start/end times, duration, cost, and delay. Additional analysis views present energy-cost breakdowns, forecast accuracy, machine-utilization heatmaps, and tariff-impact analysis.

9.3 Illustrative Output

Figure 2 shows the per-machine Gantt execution timeline produced by the dashboard for the hybrid CP-SAT schedule; Figure 3 reproduces the dashboard's published multi-strategy benchmark report; and Figure 4 presents the corresponding executive comparison charts, all discussed further in Section 12.

10 Implementation Details

10.1 Configuration Management

A centralized configuration class avoids scattering magic numbers throughout the codebase. All file paths, the global random seed, the scheduling slot duration (15 minutes) and horizon (96 slots, i.e., 24 hours per cycle), the tariff table, and the forecasting hyperparameters are declared in one place:

```

class Config:
    # Paths
    PROJECT_ROOT = Path(__file__).resolve().parent.parent
    DATA_DIR = PROJECT_ROOT / "data"
    OUTPUT_DIR = PROJECT_ROOT / "output"
    MODELS_DIR = PROJECT_ROOT / "backend" / "models"

    # Global constants
    RANDOM_SEED = 42
    SLOT_DURATION_MIN = 15
    SCHEDULING_HORIZON_SLOTS = 96

    # Tariff structure (INR per kWh)
    TARIFF_MAX_LOAD = 12.0
    TARIFF_MED_LOAD = 8.0
    TARIFF_LIGHT_LOAD = 5.0

    # Forecasting
    LAG_STEPS = [1, 2, 4, 96]
    ROLLING_WINDOWS = [4, 96]

```

10.2 Pipeline Orchestration

The main orchestration script sequences the four pipeline phases:

```

# Phase 1: Preprocessing
energy_df = loader.load_energy_data()
energy_df = cleaner.clean_energy_data(energy_df)
engineered_df = engineer.transform(energy_df)

# Phase 2: Forecasting
forecaster = MultiQuantileForecaster()
forecaster.fit(X_train, y_train)
predictions_df = forecaster.predict(X_test)

# Phase 3: Scheduling
schedules = {
    'fcfs': solve_fcfs_scheduler(...),
    'edf': solve_edf_scheduler(...),
    'fd_pdots_det': solve_proposed_scheduler(...),
    'fd_pdots_robust': solve_robust_scheduler(...),
    'makespan': solve_makespan_scheduler(...),
    'hybrid': solve_hybrid_scheduler(...),
}

# Phase 4: KPI Calculation and Reporting
for name, schedule_df in schedules.items():
    kpis = compute_schedule_kpis(schedule_df, jobs_df, machines_df,
                                predictions_df)

```

10.3 Error Handling and Logging

The system implements structured logging throughout the pipeline, recording pipeline start and end times, phase-completion status, data-quality warnings, performance metrics, and error traces for debugging. This proved valuable during development for isolating the specific pipeline stage

responsible for a given anomaly, particularly for the forecasting-stage defects discussed in Section 13.

11 Experimental Setup

The results reported in Section 12 use the synthetic benchmark instance summarized in Table 5: an energy dataset of 35,040 fifteen-minute records over calendar year 2018, and a job/machine dataset of 180 jobs across 8 machines, deliberately constructed with 75% peak-hour job arrivals so that energy-aware scheduling behavior is clearly exercised. As discussed in Section 13, this is a single synthetic instance rather than a validated multi-instance benchmark, which is an explicit limitation of the present study.

12 Results

12.1 Forecasting Performance

On the held-out chronological test split, the p50 energy forecast achieved a mean absolute error (MAE) of 19.85 kWh, a root-mean-square error (RMSE) of 35.65 kWh, and a mean absolute percentage error (MAPE) of 32.24%. Empirical coverage of the p10 quantile was 99.99% and of the p90 quantile was 48.46%, both far from their nominal 10% and 90% targets, indicating substantial miscalibration of the forecast uncertainty interval. In 13.14% of test rows the p50 prediction exceeded the p90 prediction, a violation of the required ordering $\hat{y}_{0.10} \leq \hat{y}_{0.50} \leq \hat{y}_{0.90}$ that makes the associated uncertainty interval mathematically invalid for that subset of predictions. The power-factor model produced a constant p50 prediction of 8.0, whereas power factor is physically bounded to approximately $[0, 1]$; the model does not currently enforce this bound. The 48-hour recursive future forecast produced p50 values in the range of approximately 3.04–3.71 kWh against a historical mean of 27.39 kWh and a historical maximum of 157.18 kWh, confirming that the recursive forecast is not currently usable in its present form, for the reasons discussed in Section 13.

12.2 Scheduling and KPI Comparison

Table 9 reports the full ten-metric comparison across all six scheduling strategies on the shared benchmark instance. Relative to FCFS, the hybrid CP-SAT schedule reduces energy cost by 32.4% and peak grid load by 42.4%, and reduces carbon emissions by 28.9%, while makespan increases by 0.5%, average waiting time increases by 73.1%, and machine utilization decreases by 0.24 percentage points. All six strategies achieve 100% on-time completion with zero late jobs on this instance.

Table 9: Full KPI comparison across all six scheduling strategies (single synthetic benchmark instance).

Metric	FCFS Baseline	EDF	Makespan Greedy	FD-PDTS Robust (p90)	FD-PDTS Det. (p50)	Hybrid CP-SAT
Total energy cost (INR)	186,096.29	185,686.40	186,096.29	167,723.41	167,723.41	125,832.53
Peak grid load (kW)	778.28	778.28	778.28	778.28	778.28	448.28
Makespan (h)	49.25	49.00	49.25	49.25	49.25	49.50
Machine utilization (%)	47.78	48.02	47.78	47.78	47.78	47.54
Average waiting time (min)	54.6	61.2	54.6	89.3	89.3	94.5
Total idle time (min)	12,345.0	12,225.0	12,345.0	12,345.0	12,345.0	12,465.0
Total delay (min)	0.0	0.0	0.0	0.0	0.0	0.0
Late jobs	0	0	0	0	0	0
On-time completion (%)	100.00	100.00	100.00	100.00	100.00	100.00
Carbon emissions (tCO ₂)	579.15	606.38	579.15	521.39	521.39	411.78



Figure 2: Machine scheduling Gantt timeline for the hybrid CP-SAT (warm-start optimized) schedule, coloured by job priority.

Full published benchmark report across all heuristics, greedy, and robust scheduling solvers.

Metric	FCFS Baseline	Earliest Deadline First	Makespan Greedy	Robust Greedy (p90)	Deterministic Greedy (p50)	Hybrid CP-SAT (Warm-started)
Total Energy Cost (INR)	INR 186096.29	INR 185686.40	INR 186096.29	INR 167723.41	INR 167723.41	INR 125832.53
Peak Grid Load (kW)	778.28 kW	778.28 kW	778.28 kW	778.28 kW	778.28 kW	448.28 kW
Makespan (hours)	49.25 hrs	49.00 hrs	49.25 hrs	49.25 hrs	49.25 hrs	49.50 hrs
Machine Utilization (%)	47.78%	48.02%	47.78%	47.78%	47.78%	47.54%
Average Waiting Time (min)	54.6 min	61.2 min	54.6 min	89.3 min	89.3 min	94.5 min
Total Idle Time (min)	12345.0 min	12225.0 min	12345.0 min	12345.0 min	12345.0 min	12465.0 min
Total Delay (min)	0.0 min	0.0 min	0.0 min	0.0 min	0.0 min	0.0 min
Late Jobs (count)	0 jobs	0 jobs	0 jobs	0 jobs	0 jobs	0 jobs
On-Time Completion (%)	100.00%	100.00%	100.00%	100.00%	100.00%	100.00%
Carbon Emissions (tCO2)	579.1537 t	606.3760 t	579.1537 t	521.3897 t	521.3897 t	411.7804 t

Figure 3: Full published benchmark report across all six scheduling strategies, as rendered by the dashboard.

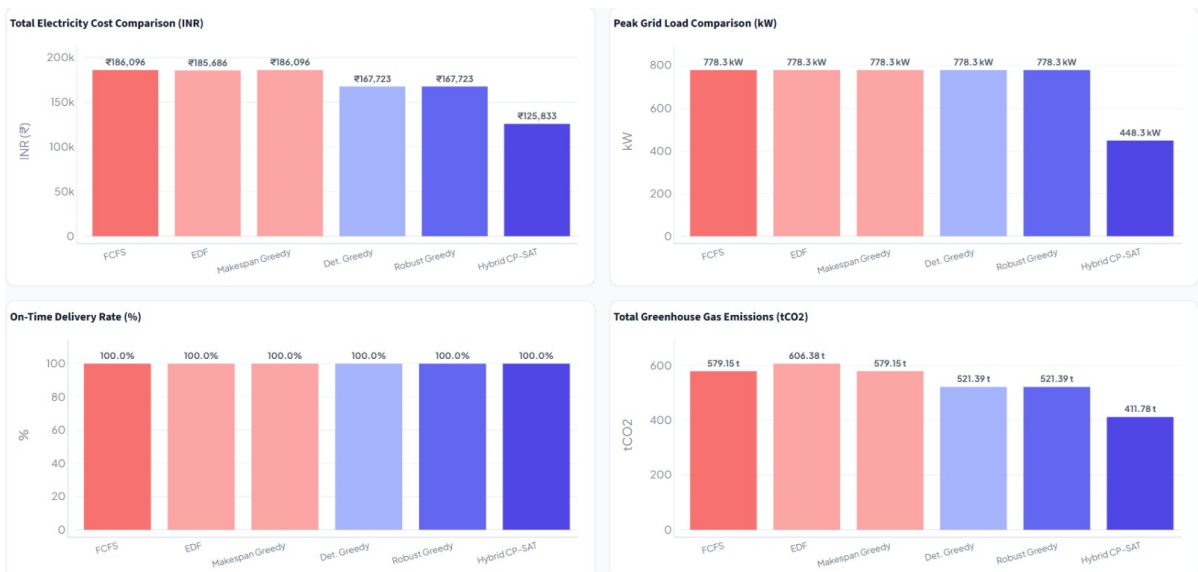


Figure 4: Executive KPI comparison charts across all six scheduling strategies.

As Table 9 shows, the Makespan Greedy strategy is numerically identical to FCFS on every reported metric, indicating it is not currently an independent makespan-minimizing method, and the FD-PDTS Robust (p90) and FD-PDTS Deterministic (p50) strategies are numerically identical to one another, indicating that the current benchmark does not demonstrate an observable difference between the robust and deterministic variants of the proposed heuristic. Both observations are discussed further, together with their likely causes, in Section 13.

13 Discussion: Validity Assessment and Limitations

An independent audit of the pipeline was conducted to assess whether the results in Section 12 could be presented as validated industrial evidence. This section reports that audit’s findings candidly.

13.1 Invalid Recursive Forecasting

Feature engineering computes usage lags at 1, 2, 4, and 96 steps, rolling statistics, and an exponentially weighted moving average, all of which are valid predictors for one-step-ahead forecasting within the historical test window. For the 48-hour recursive future forecast, however, every one of these features must be recomputed at each future step using previously predicted values rather than the original historical series. The current implementation updates only the 1-step lag recursively; lags 2, 4, and 96, all rolling statistics, and the exponentially weighted moving average remain frozen at their last historical value throughout the horizon. This is a correctness defect, not a modeling choice, and it is the principal reason the 48-hour future forecast reported in Section 12.1 should not be treated as valid.

13.2 Quantile Crossing and Miscalibration

A valid quantile model should satisfy $\hat{y}_{0.10} \leq \hat{y}_{0.50} \leq \hat{y}_{0.90}$ for every prediction. The audit found this violated in 13.14% of test rows, alongside p10/p90 empirical coverage of 99.99%/48.46% against nominal 10%/90% targets. Likely contributing causes include the compatibility of the custom XGBoost objective with the installed XGBoost version, the correctness of the supplied gradient and Hessian, and the absence of any post-hoc quantile-ordering correction such as the monotone rearrangement procedure of Chernozhukov et al. [25], which directly addresses the quantile crossing problem by sorting non-monotone estimated quantile curves into a monotone rearranged curve provably closer to the true quantile function.

13.3 Unbounded Power-Factor Output

The power-factor model produces a constant prediction of 8.0, while power factor is physically bounded to approximately $[0, 1]$. The model does not currently enforce this bound, either through output clipping, a bounded activation, or a reparameterization of the target variable.

13.4 Peak-Load Unit Inconsistency

The peak-load KPI calculation was found to combine forecasted 15-minute energy quantities (in kWh) directly with instantaneous machine power draw (in kW) without applying the necessary conversion $P_{\text{kW}} = E_{\text{kWh}}/0.25 \text{ h}$, producing a dimensionally inconsistent peak-load figure. Similarly, per-job energy cost in the schedule output combines active and setup energy, while the reported KPI total additionally includes idle energy over the full makespan, so the sum of per-job costs does

not equal the reported total energy cost; this should be corrected and clearly documented before final publication.

13.5 Single-Instance Benchmark Bias

The synthetic job/machine dataset was deliberately constructed to favor energy-aware scheduling: peak-heavy arrivals favor load shifting, heterogeneous machine power favors efficient-machine routing, and deadlines provide enough flexibility for off-peak scheduling. This design choice is appropriate for demonstrating the concept but means the reported 32.4% cost reduction and 42.4% peak-load reduction should not be presented as general industrial evidence without testing across many independently generated instances, as discussed further in Section 19.

13.6 Duplicated and Non-Differentiated Baselines

As observed directly in Table 9, the Makespan Greedy strategy is currently implemented identically to FCFS, and the FD-PDTS Robust and Deterministic variants currently produce numerically identical schedules on this benchmark instance. The former indicates that Makespan Greedy is not yet an independent baseline; the latter indicates that the heuristic robustness bias (a fixed empirical tariff-percentile shift) is not currently expressive enough to produce an observably different schedule from the deterministic variant on this instance, motivating the mathematically explicit robust reformulation discussed in Section 19.

14 Challenges Encountered and Engineering Solutions

Building the system surfaced a number of concrete engineering challenges, each with a specific resolution, summarized here for reproducibility and to document design decisions that are not otherwise visible from the final code alone.

14.1 Machine Availability Modeling

Initial models assumed each machine had a single "free pointer," akin to a single-processor queue, but real manufacturing machines follow multi-shift schedules with daytime gaps – for example, busy during a night shift, idle during the day due to other production-line commitments, and busy again during an evening shift. This was resolved by implementing the interval-based `MachineCalendar` class described in Section 7.9, which explicitly tracks busy intervals and can therefore identify and fill daytime gaps that a single-pointer model would miss.

14.2 Tariff Structure Integration

Grid tariffs change dynamically across Maximum, Medium, and Light Load categories, but simple scheduling heuristics such as FCFS and EDF do not natively consider time-varying costs. This was resolved by parameterizing tariff lookups through the centralized configuration class (Section 10.1), making the tariff-to-load-type mapping a single, auditable source of truth used consistently by both the forecasting and scheduling modules.

14.3 Forecast Horizon Boundary

Jobs can extend beyond the 48-hour forecast window used by the pipeline. A naive treatment might assign zero cost to time beyond the window, which would create a perverse incentive to defer

work indefinitely. This was resolved by implementing cyclic tariff extension, in which the forecast pattern is repeated beyond the horizon, ensuring fair and consistent cost attribution regardless of how far a job is deferred (Section 8.6).

14.4 Serialization of Custom Objective Functions

XGBoost’s custom objective functions, when implemented as stateful objects, initially failed to serialize via Joblib when defined as nested classes. This was resolved by defining the objective as a top-level class, as discussed in Section 6.5, restoring model persistence.

14.5 Data Quality and Missing Values

The underlying steel-industry energy data contained missing values, duplicate records, and inconsistent date formats. This was resolved by the comprehensive data-cleaning stage described in Section 5.1, including explicit day-first timestamp parsing, type coercion with error handling, and forward-/backward-fill imputation.

14.6 Feature Leakage in Time-Series Evaluation

An initial random train/test split introduced lookahead bias, since it could validate the model on time intervals occurring chronologically before intervals used for training. This was resolved by adopting the chronological 60/20/20 train/validation/test split described in Section 5.3.

14.7 Hyperparameter Tuning

XGBoost exposes many hyperparameters with complex interactions (tree depth, learning rate, subsampling ratios). Rather than an exhaustive search, a robust hyperparameter set (Table 7) was selected using domain knowledge and iterative testing: moderate depth to prevent overfitting, a low learning rate for careful learning, and subsampling for regularization.

15 Engineering Optimizations

Beyond correctness-oriented fixes, several performance optimizations were applied during development. Machine availability is tracked using explicit busy intervals rather than a per-slot boolean array, avoiding the need to allocate and scan an array with one entry per 15-minute slot for an entire year (525,600 entries per machine) when only a handful of busy intervals are typically active at once. Feature engineering uses vectorized pandas/NumPy operations (e.g., `shift()` for lag features) rather than explicit Python loops, which is substantially faster for a 35,040-row time series. XGBoost training is parallelized across all available CPU cores via the `n_jobs=-1` hyperparameter, since seven independent models (three targets across multiple quantiles) must otherwise be trained sequentially. Trained models are cached in memory by the dashboard layer rather than reloaded from disk on every user interaction. Finally, the CP-SAT refinement stage exploits a warm start from the FD-PDTS deterministic schedule rather than searching from an empty solution, allowing the solver to spend its fixed 60-second time budget on refinement rather than on initial feasibility discovery.

16 Testing and Validation Strategy

The system’s correctness is checked through unit tests targeting individual components – for example, verifying that tariff-array construction correctly maps each load-type category to its configured tariff value, and that KPI calculation returns sensible defaults (zero cost, zero makespan, 100% on-time completion) for an empty schedule – and through an integration test that exercises the full pipeline end to end: loading and cleaning data, generating forecasts, running each scheduling algorithm, and computing KPIs, asserting that no schedule is empty and that every computed energy cost is non-negative. Beyond these automated checks, validation additionally considers schedule feasibility (are all jobs scheduled, and are deadlines met or explicitly tracked as tardy), data consistency (do KPI-enriched schedules match the raw scheduler output), KPI correctness (do independently hand-calculated KPIs match the automated pipeline output), and forecast accuracy (do held-out predictions match actual observed values). As discussed in Section 13, the present test suite does not yet include automated checks for forecast quantile-ordering validity, recursive-forecast correctness, or KPI unit consistency; adding these is identified as a priority item in Section 19.

17 Scalability Considerations

Table 10 summarizes the current scalability envelope of the implementation. The dominant bottleneck for job count is the $O(n^2m)$ complexity of the FD-PDTS and hybrid scheduling algorithms, which evaluate every compatible machine for every job; for machine count, the bottleneck is calendar iteration cost per candidate placement; the forecast horizon is currently limited by the memory footprint of the tariff arrays used during scheduling; and end-to-end scheduling runtime is dominated by the combination of the greedy heuristic pass and the CP-SAT solver’s fixed time limit.

Table 10: Current scalability limits and their bottlenecks.

Component	Approximate limit	Bottleneck
Jobs	~1000	$O(n^2)$ scheduling algorithm
Machines	~100	Calendar iteration cost
Forecast horizon	48 hours (96 slots)	Memory for tariff arrays
Scheduling runtime	~30 seconds	Greedy pass + CP-SAT solver

Scaling strategies under consideration include partitioning jobs by deadline or machine type and scheduling each partition independently (parallel scheduling), coarse-grain scheduling of high-priority jobs followed by fine-grain scheduling of the remainder (hierarchical scheduling), incremental rather than full model retraining combined with a sliding data window (incremental forecasting), and distributing pandas operations and XGBoost training across a compute cluster using a framework such as Dask for very large job counts (distributed computation).

18 Potential Application Domains

The general problem structure addressed by this framework – flexible jobs, heterogeneous machines, time-varying energy tariffs, and deadline constraints – is common across several manufacturing and operations contexts. We note these as illustrative, plausible domains of applicability rather than as validated deployments, since the results in Section 12 are drawn from a single syn-

thetic benchmark instance and have not been tested against real operational data in any of the domains below.

18.1 Discrete and Process Manufacturing

Facilities with flexible job scheduling and material dynamic tariff exposure, such as steel and metal processing, are a natural fit for the tariff-shifting behavior modeled in Section 7.4–7.5, given their combination of high energy intensity and generally flexible (non-perishable) work-in-progress.

18.2 Complex Assembly and Fabrication

Facilities with significant machine-to-machine dependencies and long setup times, such as semiconductor fabrication, could plausibly benefit from the hybrid warm-start-plus-CP-SAT approach described in Section 7.7, since the constraint-programming refinement stage is specifically designed to improve upon a good but suboptimal greedy schedule where setup-time interactions are significant.

18.3 Regulated Batch Manufacturing

Facilities with strict compliance requirements and long batch times, such as pharmaceutical manufacturing, could plausibly benefit from the quantile-based robust scheduling approach described in Section 7.6, since it provides an explicit uncertainty band around forecasted costs rather than a single deterministic figure, which may be useful input to a batch-completion risk assessment.

18.4 Data Center Load Shifting

Facilities with dynamic electricity pricing and flexible, deferrable computational workloads, such as data centers, share the tariff-shifting structure this framework targets, and could plausibly apply an analogous forecast-driven dispatching approach to shift deferrable workloads toward lower-tariff, lower-carbon-intensity periods.

We emphasize that no domain-specific validation has been performed for any of the four applications above; they are included to motivate the general applicability of the problem formulation, not as evidence of demonstrated real-world performance.

19 Lessons Learned

19.1 Technical Lessons

Interval-based availability tracking handles real-world machine-scheduling complexity substantially better than simple pointer-based models. Quantile regression, providing confidence bands rather than single-point forecasts, is necessary infrastructure for risk-aware scheduling decisions, even though the present implementation’s quantile calibration itself requires correction (Section 13.2). Custom loss functions with internal state require careful, top-level class definitions for reliable serialization. Chronological, rather than random, train/test splitting is critical for time-series evaluation to avoid lookahead bias. Warm-starting a constraint-programming solver from a good heuristic solution is an effective way to obtain refinement within a modest, fixed time budget.

19.2 Project Management Lessons

Modular design, with each scheduling algorithm isolated as an independently callable function, enabled rapid experimentation and made it straightforward to add, remove, or benchmark individual algorithms. Centralizing configuration in a single class reduced the risk of inconsistent parameter values across modules. Comprehensive logging at each pipeline phase materially accelerated root-cause analysis when diagnosing the forecasting-stage defects described in Section 13. Validating data quality, forecast sanity, and schedule feasibility at each phase boundary, rather than only at the end of the pipeline, helps prevent a defect introduced early in the pipeline from silently propagating into and contaminating downstream results – a principle this project’s own experience with the recursive-forecast defect (Section 13.1) illustrates directly.

19.3 Domain Insights

Energy optimization and deadline compliance are often, though not always, complementary: scheduling early to meet a deadline frequently coincides with off-peak hours, with tension arising primarily when deadlines are very tight. Machine setup times create meaningful scheduling friction, since job sequencing and grouping of compatible job types materially affects total changeover overhead. Forecast uncertainty is a legitimate justification for robust optimization approaches, since an overly conservative worst-case approach sacrifices too much average-case performance, while probabilistic quantile bands can, in principle, support a better-calibrated risk/reward trade-off once the calibration issues identified in Section 13.2 are corrected.

20 Future Enhancements and Roadmap

Building on the limitations identified through the independent audit (Section 13) and the engineering experience described in Sections 14–17, the following improvements are planned, organized by priority and scope.

20.1 Priority 1: Forecasting Corrections

- Update every lag, rolling-window, and exponentially weighted moving-average feature at each future step using previously predicted values, rather than freezing them at their last historical value, so that the 48-hour recursive forecast becomes valid.
- Enforce quantile ordering, for example via the monotone rearrangement procedure of Chernozhukov et al. [25], so that $\hat{y}_{0.10} \leq \hat{y}_{0.50} \leq \hat{y}_{0.90}$ always holds, and bound the power-factor output to its physically valid range of approximately $[0, 1]$.
- Validate the custom tariff-weighted objective against a known, independently implemented quantile-loss baseline, and compare forecast accuracy against seasonal-naive, linear-regression, and standard (non-custom-objective) XGBoost baselines.

20.2 Priority 2: Scheduling Formulation Corrections

- Adopt a mathematically explicit robust or distributionally robust formulation, replacing the current fixed tariff-percentile heuristic bias with an uncertainty set or risk-adjusted objective, so that the robust and deterministic FD-PDTS variants produce demonstrably different, principled schedules.
- Extend the CP-SAT objective (Eq. 4) to directly optimize carbon emissions, peak load, and power-factor penalties, in addition to energy cost, waiting time, and tardiness.

- Model setup/changeover intervals and machine-availability calendars consistently across every scheduling strategy compared, so that the greedy and CP-SAT algorithm families share identical physical feasibility assumptions.

20.3 Priority 3: Evaluation Corrections

- Correct the kWh-to-kW unit conversion in the peak-load calculation, and provide an explicit, documented definition of the carbon-emissions units used throughout the pipeline.
- Separate active, setup, and idle energy consistently across all reported cost figures, so that per-job and aggregate cost figures reconcile exactly.
- Add automated tests for forecast quantile-ordering, recursive-forecast correctness, schedule feasibility, and KPI correctness, as identified in Section 15.

20.4 Priority 4: Experimental Strengthening

- Benchmark across many independently generated problem instances, varying job-load level, machine-fleet size, peak-load proportion, deadline tightness, and tariff spread, reporting results with confidence intervals rather than a single synthetic scenario.
- Strengthen the baseline comparison set to include shortest-processing-time (SPT), earliest-due-date (EDD), an energy-aware greedy heuristic, a full mixed-integer-programming formulation, CP-SAT without a warm start, and an oracle schedule computed with perfect future information.
- Report solver runtime, optimality gap, and random seeds for full reproducibility.

20.5 Longer-Term Roadmap

Beyond the near-term corrections above, longer-term extensions include a reinforcement-learning-based scheduler, of the kind surveyed in Section 2.3 [14, 15, 16, 17, 18, 19, 22, 23], that adapts its dispatch policy to evolving tariff patterns given sufficient production history; predictive maintenance integration that avoids scheduling jobs during high equipment-failure-risk windows; a real-time, streaming dashboard supporting dynamic schedule adjustment as new jobs arrive; a REST API layer for job submission and schedule retrieval; multi-facility coordination that distributes jobs across plants with different tariff zones; participation in grid demand-response programs; and structured sustainability/ESG reporting for carbon accounting. On the technical-debt side, planned work includes expanding automated test coverage, introducing model-version tracking for the forecasting models, containerizing the pipeline for reproducible deployment, and migrating from CSV-based storage to a relational database for concurrent access and audit-trail support.

21 Conclusion

This paper presented a complete predict-then-optimize software framework for energy-aware manufacturing scheduling, combining a tariff-weighted quantile forecasting component, a proposed robust dispatching heuristic (FD-PDTS), and a warm-started CP-SAT refinement stage, together with a twelve-metric KPI evaluation engine and an interactive dashboard. We described the system’s architecture, data schemas, feature engineering, custom loss function, scheduling formulations, and the concrete engineering challenges encountered and resolved during development, in sufficient detail for independent reproduction and extension.

On a single synthetic benchmark instance, the CP-SAT-refined schedule achieved a 32.4% energy-cost reduction and a 42.4% peak-load reduction relative to FCFS, at the cost of a substantially higher average waiting time – a genuine tradeoff that should be reported and evaluated explicitly rather than treated as an unqualified improvement. An independent audit identified concrete validity issues – an incompletely updated recursive forecast, quantile-ordering violations, an unbounded power-factor output, a dimensionally inconsistent peak-load KPI, and reliance on a single, deliberately favourable synthetic instance – that currently prevent these results from being presented as validated industrial evidence. We regard the overall architecture as a sound and extensible research and engineering contribution once these issues are corrected and the scheduling experiments are repeated across many independently generated instances with consistent physical constraints and stronger baselines, as detailed in the roadmap of Section 19. This work is presented as a prototype methodology and preliminary result set, together with a candid engineering account of its construction, rather than as a validated industrial optimization system.

Acknowledgments

The authors thank the Department of Artificial Intelligence and Data Science, Rajalakshmi Engineering College, for institutional support during this work.

Data Availability Statement

The synthetic datasets and code used to generate the reported results are available from the corresponding author upon reasonable request.

Author’s Contributions

S.S.K. conceived and supervised the study and edited the manuscript. S.R. contributed to methodology review and manuscript editing. H.K.D. implemented the forecasting pipeline and conducted the forecast validity audit. B.R. implemented the scheduling algorithms and KPI evaluation. B. implemented the dashboard and conducted the benchmark experiments. All authors read, revised, and approved the final manuscript.

Conflict of Interest

The authors declare that they have no conflict of interest.

Ethics

This study did not involve human participants, animal subjects, or identifiable personal data; no ethical approval was required.

References

- [1] Elmachoub, A. N., & Grigas, P. (2022). Smart “predict, then optimize.” *Management Science*, 68(1), 9–26. <https://doi.org/10.1287/mnsc.2020.3922>

- [2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). Association for Computing Machinery. <https://doi.org/10.1145/2939672.2939785>
- [3] Gahm, C., Denz, F., Dirr, M., & Tuma, A. (2016). Energy-efficient scheduling in manufacturing companies: A review and research framework. *European Journal of Operational Research*, 248(3), 744–757. <https://doi.org/10.1016/j.ejor.2015.07.017>
- [4] Fang, K., Uhan, N., Zhao, F., & Sutherland, J. W. (2011). A new approach to scheduling in manufacturing for power consumption and carbon footprint reduction. *Journal of Manufacturing Systems*, 30(4), 234–240. <https://doi.org/10.1016/j.jmsy.2011.08.004>
- [5] Koenker, R., & Bassett, G. (1978). Regression quantiles. *Econometrica*, 46(1), 33–50. <https://doi.org/10.2307/1913643>
- [6] Bertsimas, D., & Sim, M. (2004). The price of robustness. *Operations Research*, 52(1), 35–53. <https://doi.org/10.1287/opre.1030.0065>
- [7] Perron, L., & Furnon, V. (2024). *OR-Tools* (Version 9.x) [Computer software]. Google. <https://developers.google.com/optimization>
- [8] Ham, A., Park, M.-J., & Kim, K. M. (2021). Energy-aware flexible job shop scheduling using mixed integer programming and constraint programming. *Mathematical Problems in Engineering*, 2021, 1–12. <https://doi.org/10.1155/2021/6672719>
- [9] Park, M.-J., & Ham, A. (2022). Energy-aware flexible job shop scheduling under time-of-use pricing. *International Journal of Production Economics*, 248, 108507. <https://doi.org/10.1016/j.ijpe.2022.108507>
- [10] Terbrack, H., & Claus, T. (2025). The generalized energy-aware flexible job shop scheduling model: A constraint programming approach. *Computers & Industrial Engineering*, 204, 111065. <https://doi.org/10.1016/j.cie.2025.111065>
- [11] Xie, J., Gao, L., Peng, K., Li, X., & Li, H. (2019). Review on flexible job shop scheduling. *IET Collaborative Intelligent Manufacturing*, 1(3), 67–77. <https://doi.org/10.1049/iet-cim.2018.0009>
- [12] Dauzère-Pérès, S., Ding, J., Shen, L., & Tamssaouet, K. (2024). The flexible job shop scheduling problem: A review. *European Journal of Operational Research*. <https://doi.org/10.1016/j.ejor.2023.05.017>
- [13] Meng, L., Zhang, C., Ren, Y., Zhang, B., & Lv, C. (2020). Mixed-integer linear programming and constraint programming formulations for solving distributed flexible job shop scheduling problem. *Computers & Industrial Engineering*, 142, 106347. <https://doi.org/10.1016/j.cie.2020.106347>
- [14] Rui, Z., Zhang, X., Liu, M., Ling, L., Wang, X., Liu, C., & Sun, M. (2023). Graph reinforcement learning for flexible job shop scheduling under industrial demand response: A production and energy nexus perspective. *Journal of Manufacturing Systems*, 71. <https://doi.org/10.1016/j.jmsy.2024.06.001>

- [15] Song, W., Chen, X., Li, Q., & Cao, Z. (2022). Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Transactions on Industrial Informatics*, 19, 1600–1610. <https://doi.org/10.1109/TII.2022.3178888>
- [16] Tang, H., & Dong, J. (2024). Solving flexible job-shop scheduling problem with heterogeneous graph neural network based on relation and deep reinforcement learning. *Machines*, 12(8), 584. <https://doi.org/10.3390/machines12080584>
- [17] Zhao, C., & Deng, N. (2024). An actor-critic framework based on deep reinforcement learning for addressing flexible job shop scheduling problems. *Mathematical Biosciences and Engineering*, 21(1), 1445–1471. <https://doi.org/10.3934/mbe.2024062>
- [18] Chang, Y.-H., Liu, C.-H., & You, S. D. (2024). Scheduling for the flexible job-shop problem with a dynamic number of machines using deep reinforcement learning. *Information*, 15(2), 82. <https://doi.org/10.3390/info15020082>
- [19] Huang, D., Zhao, H., Zhang, L., & Chen, K. (2024). Learning to dispatch for flexible job shop scheduling based on deep reinforcement learning via graph gated channel transformation. *IEEE Access*, 12, 50935–50948. <https://doi.org/10.1109/ACCESS.2024.3384923>
- [20] Zhong, R. Y., Xu, X., Klotz, E., & Newman, S. T. (2017). Intelligent manufacturing in the context of Industry 4.0: A review. *Engineering*, 3(5), 616–630. <https://doi.org/10.1016/J.ENG.2017.05.015>
- [21] Kusiak, A. (2018). Smart manufacturing. *International Journal of Production Research*, 56(1–2), 508–517. <https://doi.org/10.1080/00207543.2017.1351644>
- [22] Zhang, C., Juraschek, M., & Herrmann, C. (2024). Deep reinforcement learning-based dynamic scheduling for resilient and sustainable manufacturing: A systematic review. *Journal of Manufacturing Systems*, 77, 962–989. <https://doi.org/10.1016/j.jmsy.2024.10.017>
- [23] Modrák, V., Sudhakarapandian, R., Balamurugan, A., & Soltysova, Z. (2024). A review on reinforcement learning in production scheduling: An inferential perspective. *Algorithms*, 17(8), 343. <https://doi.org/10.3390/a17080343>
- [24] Hong, T., Pinson, P., Wang, Y., Weron, R., Yang, D., & Zareipour, H. (2020). Energy forecasting: A review and outlook. *IEEE Open Access Journal of Power and Energy*, 7, 376–388. <https://doi.org/10.1109/OAJPE.2020.3029979>
- [25] Chernozhukov, V., Fernández-Val, I., & Galichon, A. (2010). Quantile and probability curves without crossing. *Econometrica*, 78(3), 1093–1125. <https://doi.org/10.3982/ECTA7880>